FINAL YEAR RESEARCH PROJECT
End Term Presentation

Automated Detection of Hardcoded Secrets in Modern Source Code Repositories using LLM-Augmented Static Analysis

Supervised By: Dr. Bhanu Pratap Rajak

Group No.: 03
Name of the Student(s) with Regd. No.:

Satyajeet Satapathy - 2241019340
Dinanath Dash - 2241004161
Avignyan Sarangi - 2241019304
Ashutosh Raj - 2241018015

Department of Computer Sc. and Engineering
Faculty of Engineering & Technology (ITER)
Siksha 'O' Anusandhan (Deemed to be) University
Bhubaneswar, Odisha

Presentation Outline

- Problem Statement & Motivations
  - Problem Statement
  - Motivations
- Literature Review
  - Existing Solutions & Limitations
- Proposed Methodology & Approach
- Key Components/Modules & Algorithms Used
- Dataset & Tools/Technologies
- Experimentation
  - System Specification
  - Experimental Results
  - Performance Analysis
- Conclusion and Future Scope
- References

Problem Statement

- Domain: DevSecOps and Software Supply Chain Security.

- Problem Statement:
  - Traditional static secret scanners lack semantic understanding, generating prohibitively high False Positive Rates by flagging safe test variables.
  - To resolve this, we engineered a two-stage hybrid scanning pipeline that uses Regex as a Stage-1 pre-filter and invokes a locally deployed, 4-bit quantized Meta Llama 3 model.
  - By analyzing a strict 11-line context window, our solution drastically reduces false alarms while maintaining total offline execution and data privacy.

Motivations

- Motivation:
  High FPR causes severe "Alert Fatigue" among security teams, delaying CI/CD pipelines and increasing the likelihood that genuine, critical vulnerabilities are overlooked in large-scale codebases.

- Primary Objective:
  Designed and implemented a two-stage hybrid scanning pipeline that drastically reduces False Positive Rates in secret detection without compromising recall capabilities.

- Measurable Objectives:
  - Evaluated baseline Regex performance against the Hybrid LLM pipeline using a quantitative Confusion Matrix.
  - Guaranteed zero-trust data privacy by executing a fine-tuned LLM entirely offline, while benchmarking the resulting inference latency trade-offs.

Literature Review

Table1: Summary of Existing Approaches and Research Limitations

Author	Methods/ Techniques Used	Dataset Used	Research Outcomes	Findings & Limitations
Schwarz (2025)	"Countermind" multi-layered security architecture using Semantic Boundary Logic (SBL) and Parameter-Space Restriction (PSR).	N/A (Conceptual/Architectural design).	Shifts LLM defenses from reactive post-hoc filtering to proactive pre-inference structural validation.	Post-hoc filters are brittle against prompt injections; models inherently struggle to distinguish trusted instructions from untrusted data.
Li et al. (2025)	Injecting vulnerability steering vectors into LLM activation layers without altering original parameters.	SARD, FFmpeg+Qemu, Reveal, REEF.	Outperformed state-of-the-art methods with a 12.86% F1-score increase on real-world datasets.	Existing representation methods focus on overall semantics instead of isolating specific vulnerability concepts.
Kim et al. (2025)	Reinforcement learning (RL) based context generator to infer contextual signals from user prompts.	SafetyInstruct, Advbench, Wildjailbreak, XSTest.	Reduces harmful responses while preventing unnecessary refusals of benign requests.	Conventional LLM frameworks fail on ambiguous tasks; extracting context is necessary to prevent bypassing safeguards.
Jin (2025)	Categorization of LLM adaptation into base application, static augmentation, RAG, fine-tuning, and hybrid fusion.	N/A (Survey paper analyzing standardized benchmarks).	Identified optimal architectural strengths (encoder vs. decoder) for different security priorities.	High token consumption, context window limits, and poor generalization remain major barriers.

Literature Review contd.

Table2: Summary of Existing Approaches and Research Limitations, contd.

Author	Methods/ Techniques Used	Dataset Used	Research Outcomes	Findings & Limitations
Dozono et al. (2025)	Empirical evaluation of 5 frontier LLMs comparing open-source vs. closed-source code performance.	CWE-Bench-Java (Open) and TS-Vuls (Closed, commercial Teamscale codebase).	Precision dropped significantly from 56% on open-source code to 34% on closed-source code.	LLM metrics are heavily inflated on open-source data due to training contamination.
Apostolidis et al. (2025)	CppCheck static analysis filtered by a fine-tuned CodeBERT vulnerability prediction model.	Big-Vul and ReVeal (C/C++ GitHub samples).	Reduced false positives, improving static analysis practicality with minimal impact on detection accuracy.	Vulnerability prediction models lack line-level granularity; static tools suffer from overwhelming false alarms.
Dozono et al. (2025)	Empirical evaluation of 5 frontier LLMs comparing open-source vs. closed-source code performance.	CWE-Bench-Java (Open) and TS-Vuls (Closed, commercial Teamscale codebase).	Precision dropped significantly from 56% on open-source code to 34% on closed-source code.	LLM metrics are heavily inflated on open-source data due to training contamination.
Muske & Khedker (2015)	Data flow analysis computing Complete-range Non-deterministic Value (cnv) variables.	Embedded C system applications (SCMS, BCM).	Cut redundant model checking calls by 49.49% and reduced false positive elimination time by 39.28%.	Speed gains were achieved at the cost of missing 2.78% of false positives due to execution path over-approximation.

Research Gap & Improvements

- Identified Limitation 1: Token Inefficiency & Latency Bottlenecks
  - Recent surveys (e.g., Jin, 2025) highlight that analyzing entire repositories using LLMs results in severe token consumption and context window exhaustion.
  - Our Engineered Solution: We utilized a "cascading filter" approach. By using deterministic Regex as a Stage-1 pre-filter, we only invoked the LLM on a strict 11-line context window, bypassing the latency constraints of pure-LLM scanners.

- Identified Limitation 2: Granularity and "Context Blindness"
  - Existing vulnerability models often evaluate overall file semantics rather than isolating specific vulnerability concepts (Li et al., 2025), and lack line-level granularity (Apostolidis et al., 2025).

Research Gap & Improvements contd.

- Our Engineered Solution: We extracted the Abstract Syntax Tree (AST) context (5 lines above and below the Regex match). Our LoRA fine-tuning specifically trained the model to evaluate variable scope and dummy test parameters within this micro-context.

- Identified Limitation 3: The Data Privacy Paradigm
  - State-of-the-art LLM security tools rely on cloud-based APIs. Sending proprietary, unredacted corporate source code to external servers (OpenAI/Anthropic) violates Zero-Trust security models and data compliance laws.
  - Our Engineered Solution: Total offline execution. We bridged this gap by deploying a 4-bit quantized Meta Llama 3 8B model locally using the Apple MLX framework, leveraging Unified Memory for secure, on-device inference.

Proposed Solution & Architecture

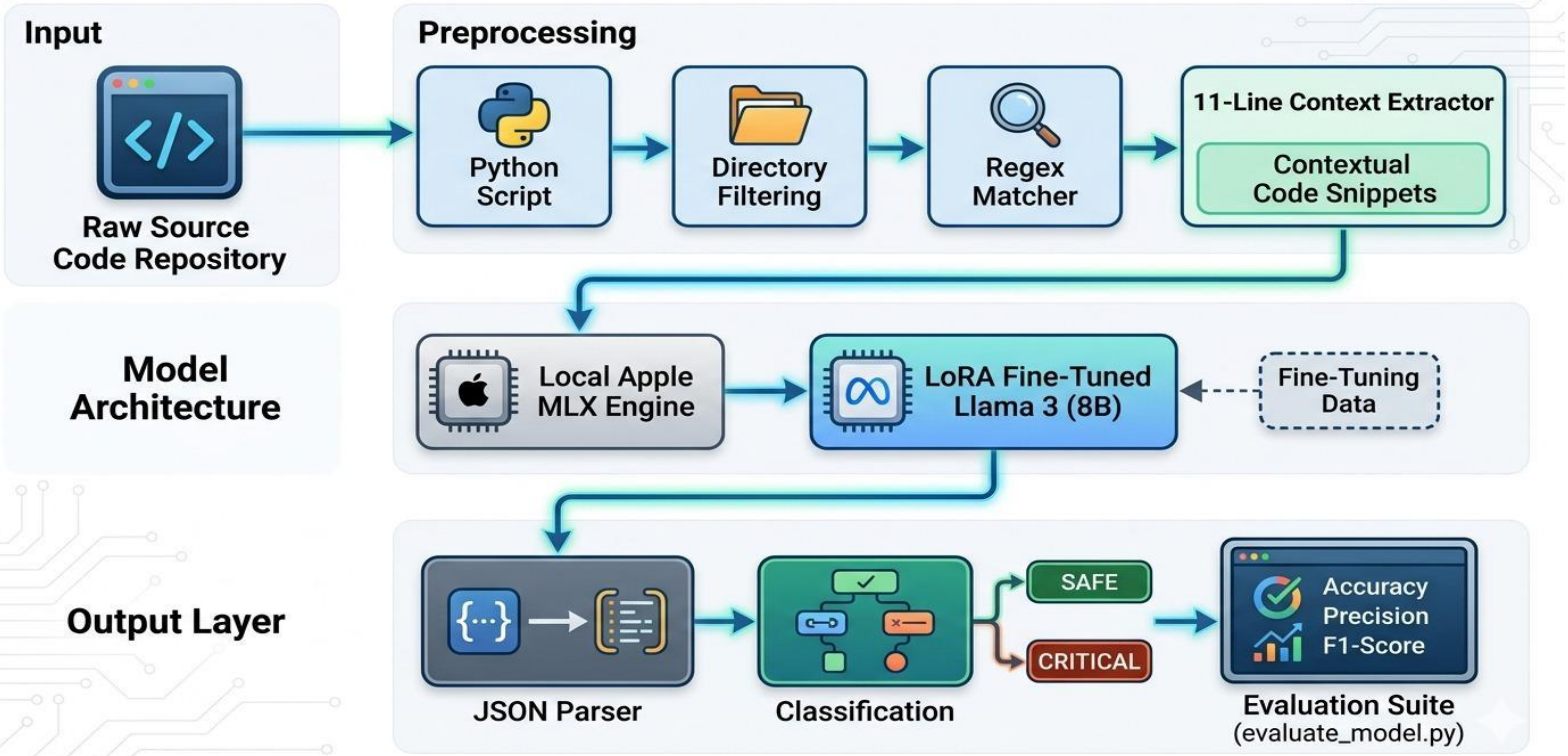

Fig. 1: Schematic Model Diagram of the Proposed Model

Dataset & Tools/Technologies

- Dataset Description:
  - Source: The open-source CredData benchmark dataset.
  - Composition: The benchmark comprises 27,000 lines of source code. Due to extreme local compute constraints, legacy baseline tools were evaluated against the full dataset, while the Hybrid LLM was rigorously benchmarked against a standardized 1,000-hit subset.

- Tools & Technologies:
  - Language & Orchestration: Python 3.10+
  - AI Model: Meta Llama 3 8B Instruct (4-bit quantized).
  - Compute Framework: Apple MLX Framework.
  - Hardware: Local execution on Apple Silicon (M-series Mac) utilizing Unified Memory for offline inference.

Experimentation: System Specifications & Parameters

- Hardware & Compute Environment: All inference executed locally on Apple Silicon (M-series Mac) utilizing Unified Memory to strictly enforce zero-trust data privacy constraints.
- Pipeline Implementation:
  - Phase 1 (Regex + 11-line AST context extraction) and Phase 2 (Apple MLX inference) were fully integrated and operationalized.
  - Model fine-tuning (4-bit quantized, LoRA at 3000 iterations) successfully stabilized strict JSON output parsing.

Experimentation: Experimental Results

- Experimental Setup: Conducted a head-to-head comparative analysis on the CredData dataset against industry-standard offline scanners (TruffleHog and Gitleaks).
- The Baseline Failure (Context Blindness):
  - TruffleHog failed entirely (Precision: 0.4301, Recall: 0.0123).
  - Gitleaks (Regex) maintained high Precision (0.8600) but suffered catastrophic False Negatives, missing 74% of actual secrets (Recall: 0.2600).
- Hybrid Model Performance:
  - Overall Accuracy: Achieved 87.0%, demonstrating stable and reliable classification across a standardized 1,000-hit evaluation subset.
  - Achieved a dominant F1-Score of 0.8723 within the test sample.
  - Precision remained highly competitive at 0.8117, while Recall surged to 0.9427.

Experimentation: Experimental Results contd.

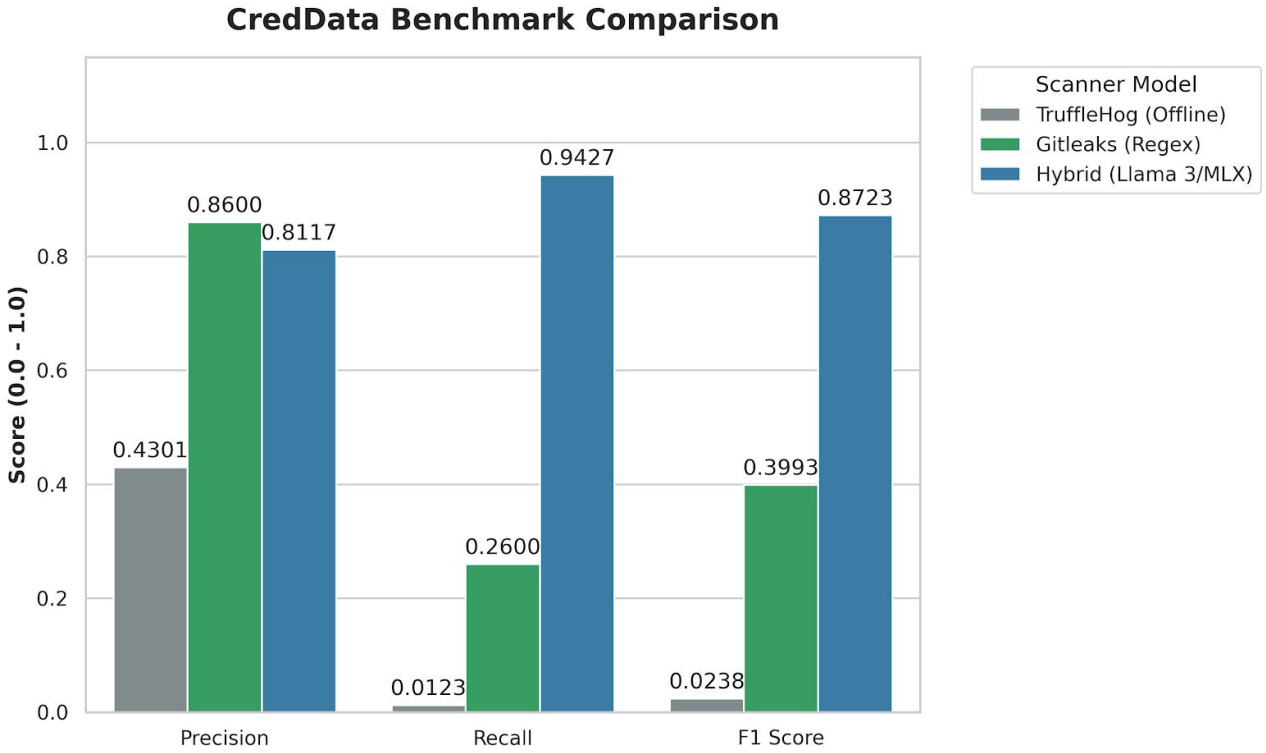

Fig. 2: Precision, Recall, and F1-Score evaluation of the Hybrid Llama 3/MLX architecture against industry baselines.

Experimentation: Performance Analysis

- The Latency Reality: While the Apple MLX framework ensured Zero-Trust privacy via local execution, executing a quantized 8B parameter model was fundamentally slower than pure C-based Regex.

- Pipeline Blocking: Executing sequential LLM inference on hundreds of initial Regex matches created a severe bottleneck, rendering it impractical for monolithic repository scans in a fast-paced CI/CD environment.

- Prompt Engineering Impact: Implementation of a strict Few-Shot Prompting template halved the initial context processing time and stabilized outputs, but the baseline per-snippet inference speed remained a critical hurdle for production deployment.

Conclusion and Future Scope

- Conclusion: The hybrid LLM-augmented pipeline successfully eliminated Regex context-blindness (achieving an 87.23% F1-Score) and guaranteed zero-trust data privacy via local MLX execution, definitively validating the core research hypothesis.

- Future Scope:
  - Cross-Dataset Validation: The model's current performance is heavily biased toward the CredData benchmark. We plan to validate generalization by testing against the FPSecretBench dataset, pending access approval from the authors.

  - Aggressive Latency Mitigation: Shift research focus from detection accuracy to operational speed. Investigate batch processing through the MLX engine or aggressive model pruning to drop per-snippet inference times to a viable DevSecOps threshold.

  - Pipeline Integration: Package the validated Python/MLX architecture into a deployable CLI tool or a standardized GitHub Action for seamless CI/CD pipeline integration.

References

[1] Schwarz, D. (2025) 'Countermind: A Multi-Layered Security Architecture for Large Language Models', arXiv preprint arXiv:2510.11837v1.

[2] Li, J. et al. (2025) 'Steering Large Language Models for Vulnerability Detection', 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP).

[3] Kim, M., Caccia, L., Shi, Z., Pereira, M., Côté, M.-A., Yuan, X., and Sordoni, A. (2025) 'Learning to Extract Context for Context-Aware LLM Inference', Microsoft Research.

[4] Jin, S. (2025) 'Large Language Models for Vulnerability Detection in Static Code Analysis: A Survey', 2025 8th International Conference on Artificial Intelligence and Big Data (ICAIBD).

[5] Dozono, K., Engesser, J., Hummelt, B., Roehm, T., and Pretschner, A. (2025) 'Should We Evaluate LLM Based Security Analysis Approaches on Open Source Systems?', 2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE).

[6] Apostolidis, G. D., Kalouptsoglou, I., Siavvas, M., Kehagias, D., and Tzovaras, D. (2025) 'AI-Enhanced Static Analysis: Reducing False Alarms Using Large Language Models', 2025 IEEE International Conference on Smart Computing (SMARTCOMP).

[7] Guo, Z. et al. (2023) 'Mitigating False Positive Static Analysis Warnings: Progress, Challenges, and Opportunities', IEEE Transactions on Software Engineering, Vol. 49, No. 12, pp. 5154-5186.

[8] Muske, T. and Khedker, U. P. (2015) 'Efficient Elimination of False Positives using Static Analysis', 2015 IEEE.

Thank you